

Les Classes et la Programmation Orientée Objet en PowerShell

Ce document est sur la programmation orientée objet (POO) native sous PowerShell (introduite avec Windows PowerShell 5.0). Il détaille la déclaration des classes et des énumérations, l'implémentation de constructeurs, les méthodes d'instance et statiques, l'héritage, ainsi que l'application concrète des classes à la modélisation d'infrastructures informatiques.

1. Fondations : Pourquoi les Classes en PowerShell ?

Depuis les premières versions de PowerShell, les administrateurs manipulent des objets .NET créés par les cmdlets (Get-Service, Get-Process). On a appris à agréger des données à l'aide de l'objet personnalisé **[PSCustomObject]**.

Cependant, à mesure que les scripts se transforment en modules d'entreprise complexes, **[PSCustomObject]** révèle ses limites structurelles.

1.1 Limites de **[PSCustomObject]** face aux Classes natives

[PSCustomObject] : Typage "Mou" et Passif]

- Pas de contrat de données strict (n'importe quelle propriété peut être ajoutée à la volée).
- Méthodes difficiles à déclarer (requiert Add-Member -MemberType ScriptMethod).
- Pas d'héritage ni de réutilisation polymorphique.
- Aucune garantie sur l'intégrité de l'objet à sa création.

[CLASSES NATIVES] : Typage Fort et Actif (PowerShell 5.0+)]

- Schéma formel et immuable (propriétés typées et contrôlées par le compilateur).
- Méthodes natives intégrées au corps de la classe.
- Constructeurs garantissant que l'objet est toujours valide dès son instanciation.
- Héritage, polymorphisme et réutilisation d'architecture.

1.2 Tableau Comparatif des Structures de Données

Critère d'évaluation	Table de Hachage (@{})	[PSCustomObject]	Classe Native (class)
Objectif premier	Dictionnaire clé / valeur	Conteneur de données tabulaire	Modèle métier complet (Données + Actions)
Contrôle du type	Aucun	Aucun (valeurs libres)	Strict (types .NET figés à la déclaration)
Méthodes actives	Non	Complexes (Add-Member)	Natives (compilées directement)
Validation à la création	Aucune	Aucune	Garantie par le constructeur (new)
Héritage	Impossible	Impossible	Supporté (class Enfant : Parent)
Performances d'accès	Standard	Standard	Optimisées par le moteur DLR (.NET)

2. Déclaration d'une Classe et Propriétés Typées

La déclaration d'une classe s'effectue via le mot-clé **class**, suivi du nom en notation PascalCase et d'un bloc d'accolades.

2.1 Propriétés d'Instance

Les propriétés définissent l'état interne de l'objet. En bonne pratique d'ingénierie, **chaque propriété doit être explicitement typée** :

```
class DisqueVirtuel {  
    # Propriétés fortement typées  
    [string]$Lettre  
    [long]$TailleOctets  
    [long]$EspaceLibreOctets  
    [bool]$EstSysteme = $false # Valeur par défaut  
}
```

Si l'utilisateur tente d'affecter une chaîne alphabétique à \$TailleOctets, PowerShell déclenche une erreur de transtypage immédiate : le schéma est garanti.

2.2 Propriétés Masquées (hidden)

Le mot-clé hidden masque une propriété lors de l'affichage console par défaut ou dans l'autocomplétion (TAB). La propriété reste néanmoins accessible par son nom exact :

```
class ServeurBase {  
    [string]$NomHote  
    hidden [string]$JetonSession # Masqué lors des Get-Member ou affichages consoles  
}
```

2.3 Propriétés Statiques (static)

Une propriété statique appartient à **la classe elle-même**, et non à une instance particulière. Tous les objets partagent la même valeur en mémoire vive :

```
class CompteurAudit {  
    static [int]$TotalServeursTraites = 0  
    [string]$NomServeur  
  
    CompteurAudit([string]$nom) {  
        $this.NomServeur = $nom  
        [CompteurAudit]::TotalServeursTraites++  
    }  
}
```

3. Les Énumérations (enum) : Bannir les Chaînes Libres

En administration système, l'une des sources majeures d'anomalies est l'usage de chaînes de texte libres pour désigner un statut (ex : "OK", "ok", "Up", "EnLigne", "Erreur").

Une **énumération** (enum) est une liste finie et fermée de valeurs admises.

3.1 Déclaration et Utilisation

Déclaration de l'énumération

```
enum StatutSante {
    Inconnu = 0
    Optimal = 1
    Critique = 2
    HorsLigne = 3
}

# Utilisation dans une classe
class MachineSupervisee {
    [string]$NomHote
    [StatutSante]$Sante = [StatutSante]::Inconnu
}
```

3.2 Contrôle Strict à l'Assignment

```
$Srv = [MachineSupervisee]::new()
$Srv.NomHote = "SRV-FILE01"

# Affectation valide (par nom ou par instance d'enum) :
$Srv.Sante = "Optimal"           # PowerShell convertit automatiquement en
[StatutSante]::Optimal
$Srv.Sante = [StatutSante]::Critique

# Affectation invalide :
$Srv.Sante = "EnPanne"
# ERREUR BLOQUANTE : Impossible de convertir "EnPanne" en [StatutSante].
```

4. Les Constructeurs (::new())

Le constructeur est la méthode spéciale exécutée **au moment précis de l'instanciation de l'objet**. Il porte obligatoirement **le même nom que la classe** et ne possède aucun type de retour.

Son rôle est d'initialiser l'objet et d'interdire la création d'instances incohérentes ou vides.

4.1 La Variable \$this

À l'intérieur d'une classe, la variable automatique **\$this** fait référence à **l'instance courante en cours de manipulation**.

4.2 Surcharge de Constructeurs (*Constructor Overloading*)

Une classe peut définir plusieurs constructeurs avec des signatures différentes :

```
class PartitionDisque {
    [string]$Lettre
    [double]$TailleGo
    [double]$LibreGo

    # 1. Constructeur par défaut (sans arguments)
    PartitionDisque() {
        $this.Lettre = "C:"
        $this.TailleGo = 100.0
        $this.LibreGo = 50.0
    }

    # 2. Constructeur paramétré (garantit les valeurs requises)
    PartitionDisque([string]$lettre, [double]$taille, [double]$libre) {
        if ($taille -le 0) {
            throw "La taille du disque doit être strictement positive."
        }
        $this.Lettre = $lettre.ToUpper()
        $this.TailleGo = [math]::Round($taille, 1)
        $this.LibreGo = [math]::Round($libre, 1)
    }
}
```

4.3 Instanciation en Mémoire

En PowerShell moderne, on instancie une classe via la méthode statique universelle **::new()** :

```
# Appel du constructeur par défaut
$P1 = [PartitionDisque]::new()

# Appel du constructeur paramétré
$P2 = [PartitionDisque]::new("D:", 500.0, 120.5)

# Instanciation invalide (déclenche immédiatement l'exception du constructeur)
$P3 = [PartitionDisque]::new("E:", -10, 0)
```

5. Les Méthodes : Associer l'Action aux Données

Dans un script classique, les données et les fonctions sont découplées :

```
$Donnees = Get-Data puis Test-Connection -Target $Donnees.IP.
```

Avec une classe, l'action fait partie intégrante de l'objet.

5.1 Règle Fondamentale des Méthodes de Classe en PowerShell

RÈGLE TECHNIQUE CRITIQUE :

Contrairement aux fonctions PowerShell standards qui émettent silencieusement leurs résultats dans le pipeline, les méthodes de classe ont un comportement calqué sur le C# :

1. Une méthode doit **explicitement déclarer son type de retour** (ou [void] si elle ne renvoie rien).
2. Tout renvoi de valeur doit obligatoirement utiliser le mot-clé **return**.
3. Aucun appel de cmdlet interne ne doit "fuir" dans la méthode. Si une commande produit une sortie non capturée, PowerShell lève une erreur de compilation ou de typage.

```
class ServeurDistant {
    [string]$NomHote
    [string]$AdresseIP
    [StatutSante]$Etat = [StatutSante]::Inconnu

    ServeurDistant([string]$nom, [string]$ip) {
        $this.NomHote = $nom
        $this.AdresseIP = $ip
    }

    # Méthode avec type de retour booléen strict
    [bool] TesterPing() {
        # [void] est obligatoire pour empêcher Test-Connection de polluer le flux
        $Reponse = Test-Connection -ComputerName $this.AdresseIP -Count 1 -Quiet

        if ($Reponse) {
            $this.Etat = [StatutSante]::Optimal
            return $true
        } else {
            $this.Etat = [StatutSante]::HorsLigne
            return $false
        }
    }
}
```

```

}

# Méthode ne renvoyant rien ([void])
[void] ReinitialiserEtat() {
    $this.Etat = [StatutSante]::Inconnu
}

# Méthode renvoyant un ratio mathématique
[double] CalculerRatioDisponibilite([int]$TotalTests, [int]$TestsReussis) {
    if ($TotalTests -eq 0) { return 0.0 }
    return [math]::Round(($TestsReussis / $TotalTests) * 100, 2)
}
}

```

5.2 Méthodes Statiques (Outils Utilitaires de Classe)

Une méthode statique s'exécute sans nécessiter d'instancier un objet préalable avec `::new()` :

```

class OutilsReseau {
    static [bool] EstIPv4Valide([string]$ip) {
        [System.Net.IPAddress]$ParsedIP = $null
        return [System.Net.IPAddress]::TryParse($ip, [ref]$ParsedIP)
    }
}

```

```

# Utilisation directe :
[OutilsReseau]::EstIPv4Valide("192.168.1.50") # Retourne $true
[OutilsReseau]::EstIPv4Valide("999.999.1.1") # Retourne $false

```

6. Héritage et Polymorphisme (class Enfant : Parent)

L'héritage permet de définir une classe de base (parent) contenant les propriétés et méthodes communes, puis de créer des classes spécialisées (enfants) qui en héritent.

6.1 Syntaxe de Dérivation

La syntaxe utilise le signe deux-points : :

```
class ClasseFille : ClasseMere
```

Classe Parente (Générique)

```
class RessourceReseau {  
    [string]$Nom  
    [string]$AdresseIP  
  
    RessourceReseau([string]$nom, [string]$ip) {  
        $this.Nom      = $nom  
        $this.AdresseIP = $ip  
    }  
  
    [string] ObtenirDescription() {  
        return "Ressource réseau : $($this.Nom) ($($this.AdresseIP))"  
    }  
}
```

Classe Dérivée 1 : Serveur Windows

```
class ServeurWindows : RessourceReseau {  
    [string]$VersionOS  
    [int]$NombreCPU  
  
    # Le constructeur enfant appelle le constructeur parent via ': base(...)'  
    ServeurWindows([string]$nom, [string]$ip, [string]$os, [int]$cpu) : base($nom, $ip) {  
        $this.VersionOS = $os  
        $this.NombreCPU = $cpu  
    }  
  
    # Redéfinition (Overriding) de méthode  
    [string] ObtenirDescription() {  
        return "Serveur Windows : $($this.Nom) [OS: $($this.VersionOS), CPU:  
$($this.NombreCPU)]"  
    }  
}
```

Classe Dérivée 2 : Équipement Réseau (Commutateur / Switch)

```
class SwitchReseau : RessourceReseau {  
    [int]$NombrePorts  
    [int]$VlanGestion  
  
    SwitchReseau([string]$nom, [string]$ip, [int]$ports, [int]$vlan) : base($nom, $ip) {  
        $this.NombrePorts = $ports  
        $this.VlanGestion = $vlan  
    }  
}
```

6.2 Le Polymorphisme en Action

Le polymorphisme permet de manipuler une collection hétérogène d'objets dérivés via leur interface commune :

```
$Inventaire = [System.Collections.Generic.List[RessourceReseau]]::new()

$Inventaire.Add([ServeurWindows]::new("SRV-AD01", "10.0.0.1", "Windows Server 2022", 8))
$Inventaire.Add([SwitchReseau]::new("SW-CORE-01", "10.0.0.254", 48, 100))

foreach ($Equipement in $Inventaire) {
    # Chaque objet appelle sa propre version de ObtenirDescription() !
    Write-Host $Equipement.ObtenirDescription() -ForegroundColor Cyan
}
```

7. Cas d'Usage Avancé : Validateurs d'Attributs Personnalisés

L'une des fonctionnalités les plus avancées des classes en administration système consiste à créer ses propres **décorateurs de validation pour les fonctions PowerShell** en héritant de `System.Management.Automation.ValidateArgumentsAttribute`.

Exemple : Création d'un validateur d'adresse MAC

```
# Déclaration de l'attribut de validation sur mesure
class ValidateMacAddressAttribute :
System.Management.Automation.ValidateArgumentsAttribute {
    [void] Validate([object]$arguments,
[System.Management.Automation.EngineIntrinsics]$engineIntrinsics) {
        $Pattern = '^([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})$'
        if ($arguments -notmatch $Pattern) {
            throw "L'adresse MAC '$arguments' est invalide. Format attendu : XX:XX:XX:XX:XX:XX
ou XX-XX-XX-XX-XX-XX."
        }
    }
}
```

```
# Utilisation directe dans une fonction avancée comme s'il s'agissait d'un attribut natif !
function Set-AdresseMacServeur {
```

```

[CmdletBinding()]
param(
    [Parameter(Mandatory = $true)]
    [ValidateMacAddress()] # Notre validateur personnalisé !
    [string]$AdresseMAC
)

Write-Host "Adresse MAC validée : $AdresseMAC" -ForegroundColor Green
}

```

8. L'Intégration des Classes dans les Modules PowerShell

L'intégration de classes dans un module d'entreprise (.psm1 / .psd1) impose une rigueur particulière en raison de l'ordre d'analyse syntaxique du moteur PowerShell.

8.1 Le Problème du Import-Module Classique

Lorsque vous écrivez :

```
Import-Module MonModuleClasses
```

Le module est importé dans la session, mais **les définitions de types de classes ne sont pas immédiatement exportées dans le scope de la session appelante**. L'utilisateur ne peut pas toujours instancier [MaClasse]::new() directement.

8.2 La Directive Maîtresse : using module

Pour charger un module et exporter **ses classes et ses enums** directement dans la session appelante avec une résolution de type au moment de la compilation :

```
# Au tout début du script consommateur (impérativement à la ligne 1) :
using module .\MonModuleClasses\MonModuleClasses.psd1
```

```
# La classe est désormais reconnue comme un type de premier rang :
$Objet = [MaClasseExportee]::new()
```

8.3 Organisation dans l'Arborescence du Module

Pour maintenir la lisibilité :

1. Déposer les définitions d'énumérations et de classes dans un sous-dossier Classes/ (ex : Classes\StatutSante.ps1, Classes\Serveur.ps1).
2. Dans le .psm1, dot-sourcer les classes **AVANT** les fonctions des dossiers Private/ et Public/ afin que les fonctions puissent utiliser les classes comme types de paramètres.
3. Déclarer explicitement les classes dans la clé ScriptsToProcess ou utiliser using module dans le manifeste.

MonModule/

```
|— MonModule.psd1
|— MonModule.psm1
|— Classes/      <--- Déclarées et dot-sourcées en premier
|   |— StatutEnum.ps1
|   |— Serveur.ps1
|— Private/
|— Public/
```

9. Synthèse des Bonnes Pratiques de Conception

1. **Typier 100 % des propriétés** : Bannir les propriétés non typées dans une classe. L'intérêt d'une classe réside dans son contrat de données strict.
2. **Neutraliser les sorties accidentelles dans les méthodes** : Toujours typer les méthodes ([void], [string], [bool]). Utiliser Out-Null, [void] ou rediriger vers \$null toute commande système appelée à l'intérieur d'une méthode pour éviter de polluer le flux de retour.
3. **Privilégier les constructeurs paramétrés** : Ne pas créer d'objets vides dont les propriétés obligatoires doivent être renseignées manuellement après coup. Le constructeur doit garantir qu'un objet instancié est immédiatement sain.
4. **Réserver les classes au domaine métier** : Continuer d'utiliser [PSCustomObject] pour les projections de données temporaires et rapides en fin de pipeline (Select-Object), et réserver les class pour les entités pérennes d'infrastructure dotées de logique active (serveurs, connexions, configurations).